

29. solar 運用 CSV logger

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	6
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	6
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	7
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	8
6.8 rclpy、rcl、rmw、DDS/RTSPS の層	8
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	9
6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか	9
6.12 天候、route、補間、時刻、単位	10
6.13 制御、状態、入力、モデル予測制御 MPC	10
7 関数・クラスを上から順に解説	10

7.1	L85 クラス SolarLoggerNode	10
7.2	L86 関数 SolarLoggerNode.__init__	12
7.3	L139 関数 SolarLoggerNode._set_float	14
7.4	L140 関数 SolarLoggerNode._set_float.handler	15
7.5	L148 関数 SolarLoggerNode._set_string	16
7.6	L149 関数 SolarLoggerNode._set_string.handler	17
7.7	L154 関数 SolarLoggerNode._set_array	17
7.8	L155 関数 SolarLoggerNode._set_array.handler	18
7.9	L163 関数 SolarLoggerNode._clean_float	19
7.10	L170 関数 SolarLoggerNode._write_row	19
7.11	L183 関数 SolarLoggerNode.destroy_node	21
7.12	L192 関数 main	22
7.13	設定値と入力パラメータ	23
7.14	ROS topic I/O	23
8	処理の流れ	23
9	このファイルの位置づけ	23

1 対象

項目	内容
ファイル	mpc_solarcar/solar_logger_node.py
ソース SHA-256	6ad5861c795593dc8a6cbd661335f2d379ddf42987433a40f1805c49884b5e2e
種別	Python
区分	runtime node
役割	vehicle、planner、weather、calib、system、raw telemetry を一つの時刻行へ集約して CSV に書く。
起動文脈	運用ログの最終集約点。

2 このファイルを読む理由

vehicle、planner、weather、calib、system、raw telemetry を一つの時刻行へ集約して CSV に書く。

- topic 名と CSV 列名の対応を内部辞書で持つ。
- planner/env、planner/metrics、planner/status も記録する。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/live_launch.py
- launch/solar_measurement.launch.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
}

class SolarLoggerNode(Node):
    def __init__(self):
        super().__init__("solar_logger_node")
        self.declare_parameter("log_dir", "outputs/logs")
        self.declare_parameter("file_prefix", "solar_live")
```

```

self.declare_parameter("log_rate_hz", 2.0)
self.declare_parameter("flush_every_rows", 1)
self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")

log_dir = os.fspath(self.get_parameter("log_dir").value)
prefix = str(self.get_parameter("file_prefix").value)
rate_hz = max(0.1, float(self.get_parameter("log_rate_hz").value))
self.flush_every_rows = max(1, int(self.get_parameter("flush_every_rows").value))
self.rows_since_flush = 0

os.makedirs(log_dir, exist_ok=True)
stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
self.log_path = os.path.join(log_dir, f"{prefix}_{stamp}.csv")

```

5 主要構造の読み取り

主要クラスは SolarLoggerNode。主要関数は handler, handler, handler, destroy_node, main。ROS パラメータ宣言は 6 件。ROS I/O は publisher 0 件、subscription 6 件。

5.1 主要クラス・関数

行	種別	名前	補足
85	class	SolarLoggerNode	bases: Node
86	function	__init__	args: self
139	function	_set_float	args: self, field
140	function	handler	args: msg
148	function	_set_string	args: self, field
149	function	handler	args: msg
154	function	_set_array	args: self, fields
155	function	handler	args: msg
163	function	_clean_float	args: value
170	function	_write_row	args: self
183	function	destroy_node	args: self
192	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
---	----

```

11  FLOAT_TOPICS に {'speed_kmh': '/vehicle/speed_kmh', 's_kmh': '/vehicle/s_kmh',
    'altitude_m': '/vehicle/altitude_m', 'grade_pct': '/vehicle/grade', 'batt_soc': '/vehicle/batt_soc',
    'batt_temp_c': '/vehicle/batt_temp_c', 'batt_current_a': '/vehicle/batt_current_a',
    'batt_voltage_v': '/vehicle/batt_voltage_v', 'solar_power_w': '/vehicle/solar_power_w',
    'headwind_meas_ms': '/weather/headwind_meas_ms', 'headwind_corrected_ms':
    '/weather/headwind_corrected_ms', 'wind_speed_ms': '/weather/wind_speed_ms',
    'wind_dir_deg': '/weather/wind_dir_deg', 'course_deg': '/weather/course_deg',
    'speed_cmd_kmh': '/planner/speed_cmd', 'upper_speed_cmd_kmh': '/planner/upper_speed_cmd',
    'throttle_cmd_pct': '/planner/throttle_cmd_pct', 'calib_solar_gain': '/calib/solar_gain',
    'calib_drive_power_gain': '/calib/drive_power_gain', 'calib_aux_power_w': '/calib/aux_power_w',
    'system_health': '/system/health'} の結果を代入する。
35  FLOAT64_TOPICS に {'solar_source_ts_unix': '/telemetry/solar_source_ts_unix',
    'chase_source_ts_unix': '/telemetry/chase_source_ts_unix'} の結果を代入する。
40  STRING_TOPICS に {'drive_mode': '/planner/drive_mode', 'system_state': '/system/state',
    'system_diag': '/system/diag', 'mpc_state': '/system/mpc_state', 'telemetry_bridge_status':
    '/telemetry/bridge_status', 'wind_correction_status': '/weather/wind_correction_status',
    'weather_fetch_status': '/weather/fetch_status', 'autocal_status': '/calib/status', 'raw_solar':
    '/telemetry/raw_solar', 'raw_chase': '/telemetry/raw_chase'} の結果を代入する。
53  ARRAY_TOPICS に {'/planner/status': ['planner_soc', 'planner_temp_c', 'planner_s_kmh',
    'planner_step', 'planner_sec_to_next', 'planner_control_stop_hold', 'planner_control_stop_remaining_sec',
    'planner_control_stop_completed_count'], '/planner/metrics': ['model_pack_voltage_v', 'model_pack_current_a',
    'model_soc', 'model_motor_power_w', 'model_motor_current_a', 'model_pv_power_w', 'model_speed_kmh',
    'model_mech_power_w', 'model_pack_power_w'], '/planner/env': ['env_poa_wm2', 'env_cell_temp_c',
    'env_ambient_temp_c', 'env_grade_pct', 'env_headwind_ms']} の結果を代入する。
85  クラス SolarLoggerNode を定義する。
192  関数 main を定義する。

```

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import csv</code>	CSV の逐次読込・逐次書込を行うため。このファイル内での主な使用位置は L133。
2	<code>import math</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L109, L144, L158, L168。
3	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L95, L101, L103。
4	<code>from datetime import datetime, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L102, L172。
6	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L193, L196, L199。
7	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L85。
8	<code>from std_msgs import Float32, Float32MultiArray, Float64, String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L115, L117, L119, L121, L126, L129。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
```

```
counter = Counter()
counter.increment()
```

‘`super().__init__(...)`’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘`create_publisher`’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘`self.step_solar`’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘`_`’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘`_base_csv`’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は MultiThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosviz をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosviz は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L85 クラス SolarLoggerNode

- 行範囲: L85–L189
- 所属: module 直下
- 定義: `SolarLoggerNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_set_float` を定義する。
3. 関数 `_set_string` を定義する。
4. 関数 `_set_array` を定義する。
5. 関数 `_clean_float` を定義する。
6. 関数 `_write_row` を定義する。
7. 関数 `destroy_node` を定義する。

代表コード断片

```
class SolarLoggerNode(Node):
    def __init__(self):
        super().__init__("solar_logger_node")
        self.declare_parameter("log_dir", "outputs/logs")
        self.declare_parameter("file_prefix", "solar_live")
        self.declare_parameter("log_rate_hz", 2.0)
        self.declare_parameter("flush_every_rows", 1)
        self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
        self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")

        log_dir = os.fspath(self.get_parameter("log_dir").value)
        prefix = str(self.get_parameter("file_prefix").value)
        rate_hz = max(0.1, float(self.get_parameter("log_rate_hz").value))
        self.flush_every_rows = max(1, int(self.get_parameter("flush_every_rows").value))
        self.rows_since_flush = 0

        os.makedirs(log_dir, exist_ok=True)
        stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.log_path = os.path.join(log_dir, f"{prefix}_{stamp}.csv")

        array_fields = [field for fields in ARRAY_TOPICS.values() for field in fields]
        self.fields = ["t_ros_sec", "t_wall_utc"] + list(FLOAT_TOPICS) + list(FLOAT64_TOPICS)
    + [
        "output_speed_cmd_kmh",
    ] + list(String_TOPICS) + ["output_drive_mode"] + array_fields
        self.latest = {field: math.nan for field in self.fields}
        for field in ["t_wall_utc", *String_TOPICS, "output_drive_mode"]:
            self.latest[field] = ""

        self._topic_subscriptions = []
        for field, topic in FLOAT_TOPICS.items():
```

```

        self._topic_subscriptions.append(self.create_subscription(Float32, topic, self.
_set_float(field), 10))
        for field, topic in FLOAT64_TOPICS.items():
            self._topic_subscriptions.append(self.create_subscription(Float64, topic, self.
_set_float(field), 10))
        for field, topic in STRING_TOPICS.items():
            self._topic_subscriptions.append(self.create_subscription(String, topic, self.
_set_string(field), 10))
...

```

7.2 L86 関数 SolarLoggerNode.__init__

- 行範囲: L86–L137
- 所属: SolarLoggerNode
- 定義: `__init__(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DictWriter, `__init__`, `_set_array`, `_set_float`, `_set_string`, `append`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `flush`, `fspath`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `array_fields`, `field`, `fields`, `log_dir`, `output_mode_topic`, `output_speed_topic`, `prefix`, `rate_hz`, `stamp`, `topic`
- 読み取る主なインスタンス属性: `self._set_array`, `self._set_float`, `self._set_string`, `self._topic_subscriptions`, `self._write_row`, `self.create_subscription`, `self.create_timer`, `self.csv_file`, `self.declare_parameter`, `self.fields`, `self.get_logger`, `self.get_parameter`, `self.latest`, `self.log_path`, `self.writer`
- 更新する主なインスタンス属性: `self._topic_subscriptions`, `self.csv_file`, `self.fields`, `self.flush_every_rows`, `self.latest`, `self.log_path`, `self.rows_since_flush`, `self.timer`, `self.writer`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 5、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。

8. `log_dir` に `os.fspath(self.get_parameter('log_dir').value)` の結果を代入する。
9. `prefix` に `str(self.get_parameter('file_prefix').value)` の結果を代入する。
10. `rate_hz` に `max(0.1, float(self.get_parameter('log_rate_hz').value))` の結果を代入する。
11. `self.flush_every_rows` に `max(1, int(self.get_parameter('flush_every_rows').value))` の結果を代入する。
12. `self.rows_since_flush` に 0 の結果を代入する。
13. `os.makedirs(...)` を実行する。
14. `stamp` に `datetime.now().strftime('%Y%m%d_%H%M%S')` の結果を代入する。
15. `self.log_path` に `os.path.join(log_dir, f'{prefix}_{stamp}.csv')` の結果を代入する。
16. `array_fields` に `[field for fields in ARRAY_TOPICS.values() for field in fields]` の結果を代入する。
17. `self.fields` に `['t_ros_sec', 't_wall_utc'] + list(FLOAT_TOPICS) + list(FLOAT64_TOPICS) + ['output_speed_cmd_kmh'] + list(STRING_TOPICS) + ['output_drive_mode'] + array_fields` の結果を代入する。
18. `self.latest` に `{field: math.nan for field in self.fields}` の結果を代入する。

`_drive_mode'` を順に走査し、各要素を `field` に入れて処理する。

19. `self.latest[field]` に” の結果を代入する。
20. `self._topic_subscriptions` に [] の結果を代入する。
21. `FLOAT_TOPICS.items()` を順に走査し、各要素を `(field, topic)` に入れて処理する。
22. `self._topic_subscriptions.append(...)` を実行する。
23. `FLOAT64_TOPICS.items()` を順に走査し、各要素を `(field, topic)` に入れて処理する。
24. `self._topic_subscriptions.append(...)` を実行する。
25. `STRING_TOPICS.items()` を順に走査し、各要素を `(field, topic)` に入れて処理する。
26. `self._topic_subscriptions.append(...)` を実行する。
27. `ARRAY_TOPICS.items()` を順に走査し、各要素を `(topic, fields)` に入れて処理する。
28. `self._topic_subscriptions.append(...)` を実行する。
29. `output_speed_topic` に `str(self.get_parameter('output_speed_topic').value)` の結果を代入する。
30. `output_mode_topic` に `str(self.get_parameter('output_drive_mode_topic').value)` の結果を代入する。
31. `self._topic_subscriptions.append(...)` を実行する。
32. `self._topic_subscriptions.append(...)` を実行する。
33. `self.csv_file` に `open(self.log_path, 'w', encoding='utf-8', newline='')` の結果を代入する。
34. `self.writer` に `csv.DictWriter(self.csv_file, fieldnames=self.fields)` の結果を代入する。
35. `self.writer.writeheader(...)` を実行する。
36. `self.csv_file.flush(...)` を実行する。
37. `self.timer` に `self.create_timer(1.0 / rate_hz, self._write_row)` の結果を代入する。
38. `self.get_logger().info(...)` を実行する。

代表コード断片

```
def __init__(self):
    super().__init__("solar_logger_node")
    self.declare_parameter("log_dir", "outputs/logs")
    self.declare_parameter("file_prefix", "solar_live")
    self.declare_parameter("log_rate_hz", 2.0)
    self.declare_parameter("flush_every_rows", 1)
    self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
    self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")

    log_dir = os.fspath(self.get_parameter("log_dir").value)
    prefix = str(self.get_parameter("file_prefix").value)
    rate_hz = max(0.1, float(self.get_parameter("log_rate_hz").value))
    self.flush_every_rows = max(1, int(self.get_parameter("flush_every_rows").value))
    self.rows_since_flush = 0

    os.makedirs(log_dir, exist_ok=True)
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    self.log_path = os.path.join(log_dir, f"{prefix}_{stamp}.csv")

    array_fields = [field for fields in ARRAY_TOPICS.values() for field in fields]
    self.fields = ["t_ros_sec", "t_wall_utc"] + list(FLOAT_TOPICS) + list(FLOAT64_TOPICS)
) + [
    "output_speed_cmd_kmh",
] + list(String_TOPICS) + ["output_drive_mode"] + array_fields
self.latest = {field: math.nan for field in self.fields}
for field in ["t_wall_utc", *String_TOPICS, "output_drive_mode"]:
    self.latest[field] = ""

    self._topic_subscriptions = []
    for field, topic in FLOAT_TOPICS.items():
        self._topic_subscriptions.append(self.create_subscription(Float32, topic, self._set_float(field), 10))
    for field, topic in FLOAT64_TOPICS.items():
        self._topic_subscriptions.append(self.create_subscription(Float64, topic, self._set_float(field), 10))
    for field, topic in String_TOPICS.items():
        self._topic_subscriptions.append(self.create_subscription(String, topic, self._set_string(field), 10))
    for topic, fields in ARRAY_TOPICS.items():
...

```

7.3 L139 関数 SolarLoggerNode._set_float

- 行範囲: L139–L146
- 所属: SolarLoggerNode
- 定義: `_set_float(self, field)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: handler
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.latest`
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `handler` を定義する。
2. `handler` を返す。

代表コード断片

```
def _set_float(self, field):
    def handler(msg):
        try:
            self.latest[field] = float(msg.data)
        except (TypeError, ValueError):
            self.latest[field] = math.nan

    return handler
```

7.4 L140 関数 `SolarLoggerNode._set_float.handler`

- 行範囲: L140–L144
- 所属: `SolarLoggerNode._set_float`
- 定義: `handler(msg)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.latest`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. self.latest[field] に float(msg.data) の結果を代入する。
3. (TypeError, ValueError) を捕捉した場合:
4. self.latest[field] に math.nan の結果を代入する。

代表コード断片

```
def handler(msg):  
    try:  
        self.latest[field] = float(msg.data)  
    except (TypeError, ValueError):  
        self.latest[field] = math.nan
```

7.5 L148 関数 SolarLoggerNode._set_string

- 行範囲: L148–L152
- 所属: SolarLoggerNode
- 定義: _set_string(self, field)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: replace, str
- 戻り値の要点: handler
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self.latest
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 関数 handler を定義する。
2. handler を返す。

代表コード断片

```
def _set_string(self, field):  
    def handler(msg):  
        self.latest[field] = str(msg.data).replace("\r", "").replace("\n", "\\n")  
  
    return handler
```


7.6 L149 関数 SolarLoggerNode._set_string.handler

- 行範囲: L149–L150
- 所属: SolarLoggerNode._set_string
- 定義: handler(msg)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: replace, str
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self.latest
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. self.latest[field] に str(msg.data).replace('r', '').replace('\n', '\\n') の結果を代入する。

代表コード断片

```
def handler(msg):  
    self.latest[field] = str(msg.data).replace("\r", "").replace("\n", "\\n")
```

7.7 L154 関数 SolarLoggerNode._set_array

- 行範囲: L154–L160
- 所属: SolarLoggerNode
- 定義: _set_array(self, fields)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: enumerate, float, len, list
- 戻り値の要点: handler
- 主なローカル代入名: field, idx, values
- 読み取る主なインスタンス属性: self.latest
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 関数 `handler` を定義する。
2. `handler` を返す。

代表コード断片

```
def _set_array(self, fields):  
    def handler(msg):  
        values = list(msg.data)  
        for idx, field in enumerate(fields):  
            self.latest[field] = float(values[idx]) if idx < len(values) else math.nan  
  
    return handler
```

7.8 L155 関数 `SolarLoggerNode._set_array.handler`

- 行範囲: L155–L158
- 所属: `SolarLoggerNode._set_array`
- 定義: `handler(msg)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `enumerate`, `float`, `len`, `list`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `field`, `idx`, `values`
- 読み取る主なインスタンス属性: `self.latest`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `values` に `list(msg.data)` の結果を代入する。
2. `enumerate(fields)` を順に走査し、各要素を `(idx, field)` に入れて処理する。
3. `self.latest[field]` に `float(values[idx]) if idx < len(values) else math.nan` の結果を代入する。

代表コード断片

```
def handler(msg):  
    values = list(msg.data)  
    for idx, field in enumerate(fields):  
        self.latest[field] = float(values[idx]) if idx < len(values) else math.nan
```

7.9 L163 関数 SolarLoggerNode._clean_float

- 行範囲: L163–L168
- 所属: SolarLoggerNode
- 定義: `_clean_float(value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: `value if math.isfinite(value) else ""`
- 主なローカル代入名: `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- `@` で始まる行は、定義した関数を別の関数へ渡して加工する `decorator` である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `value` に `float(value)` の結果を代入する。
3. (`TypeError`, `ValueError`) を捕捉した場合:
4. `""` を返す。
5. `value if math.isfinite(value) else ""` を返す。

代表コード断片

```
def _clean_float(value):
    try:
        value = float(value)
    except (TypeError, ValueError):
        return ""
    return value if math.isfinite(value) else ""
```

7.10 L170 関数 SolarLoggerNode._write_row

- 行範囲: L170–L181
- 所属: SolarLoggerNode
- 定義: `_write_row(self)`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_clean_float`, `flush`, `get`, `get_clock`, `isoformat`, `now`, `str`, `writerow`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `field`, `row`, `string_fields`
- 読み取る主なインスタンス属性: `self._clean_float`, `self.csv_file`, `self.fields`, `self.flush_every_rows`, `self.get_clock`, `self.latest`, `self.rows_since_flush`, `self.writer`
- 更新する主なインスタンス属性: `self.rows_since_flush`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.latest['t_ros_sec']` に `self.get_clock().now().nanoseconds / 1000000000.0` の結果を代入する。
2. `self.latest['t_wall_utc']` に `datetime.now(timezone.utc).isoformat()` の結果を代入する。
3. `row` に `{}` の結果を代入する。
4. `string_fields` に `{'t_wall_utc', *STRING_TOPICS, 'output_drive_mode'}` の結果を代入する。
5. `self.fields` を順に走査し、各要素を `field` に入れて処理する。
6. `row[field]` に `str(self.latest.get(field, "")) if field in string_fields else self._clean_float(self.latest.get(field))` の結果を代入する。
7. `self.writer.writerow(...)` を実行する。
8. `self.rows_since_flush` を `Add` で更新する。
9. 条件 `self.rows_since_flush >= self.flush_every_rows` を判定し、真なら内部処理を行う。
10. `self.csv_file.flush(...)` を実行する。
11. `self.rows_since_flush` に 0 の結果を代入する。

代表コード断片

```
def _write_row(self):
    self.latest["t_ros_sec"] = self.get_clock().now().nanoseconds / 1.0e9
    self.latest["t_wall_utc"] = datetime.now(timezone.utc).isoformat()
    row = {}
    string_fields = {"t_wall_utc", *STRING_TOPICS, "output_drive_mode"}
    for field in self.fields:
        row[field] = str(self.latest.get(field, "")) if field in string_fields else self
        ._clean_float(self.latest.get(field))
    self.writer.writerow(row)
    self.rows_since_flush += 1
    if self.rows_since_flush >= self.flush_every_rows:
        self.csv_file.flush()
        self.rows_since_flush = 0
```

7.11 L183 関数 SolarLoggerNode.destroy_node

- 行範囲: L183–L189
- 所属: SolarLoggerNode
- 定義: `destroy_node(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `close`, `destroy_node`, `flush`, `super`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.csv_file`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `self.csv_file.flush(...)` を実行する。
3. `self.csv_file.close(...)` を実行する。
4. `Exception` を捕捉した場合:
5. `Pass` 文を実行する。
6. `super().destroy_node(...)` を実行する。

代表コード断片

```
def destroy_node(self):
    try:
        self.csv_file.flush()
        self.csv_file.close()
    except Exception:
        pass
    super().destroy_node()
```

7.12 L192 関数 main

- 行範囲: L192–L199
- 所属: module 直下
- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SolarLoggerNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `SolarLoggerNode()` の結果を代入する。
3. 例外処理を伴う `try` ブロックを実行する。
4. `rclpy.spin(...)` を実行する。
5. 成否にかかわらず `finally` で:
6. `node.destroy_node(...)` を実行する。
7. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main():
    rclpy.init()
    node = SolarLoggerNode()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

7.13 設定値と入力パラメータ

行	名前	既定値・補足
88	log_dir	outputs/logs
89	file_prefix	solar_live
90	log_rate_hz	2.0
91	flush_every_rows	1
92	output_speed_topic	/vehicle/speed_cmd_kmh
93	output_drive_mode_topic	/vehicle/drive_mode_cmd

7.14 ROS topic I/O

行	種別	topic	callback / 補足
115	subscription	topic	self._set_float(field)
117	subscription	topic	self._set_float(field)
119	subscription	topic	self._set_string(field)
121	subscription	topic	self._set_array(fields)
126	subscription	output_speed_topic	self._set_float('output_speed_cmd_kmh')
129	subscription	output_mode_topic	self._set_string('output_drive_mode')

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_logger_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。